

Exercise 4 – Stock Price Simulation in the Black–Scholes Model

Tudor Gulin

June 15, 2026

1 Introduction

In this project I implemented a simulation framework for the stock price process in the Black–Scholes model, corresponding to Exercise 4 from the project list. The reference algorithm is Algorithm 5.1 from the book *Monte Carlo Methods and Models in Finance* by R. Korn, E. Korn, and G. Kroisandt.

The objective of the project is to simulate stock-price paths on a finite time interval $[0, T]$ for different choices of the drift function μ and the volatility function σ , and to visualize the resulting trajectories and summary statistics.

My implementation contains:

- two drift functions $\mu(t, S)$;
- two volatility functions $\sigma(t, S)$;
- two simulation schemes:
 1. the Euler–Maruyama discretization;
 2. a book-style exponential update corresponding to the one-dimensional version of Algorithm 5.1.
- an interactive Streamlit application for visualization and comparison.

2 Mathematical Model

The stock price process $(S_t)_{t \geq 0}$ is modeled by the stochastic differential equation

$$dS_t = \mu(t, S_t) S_t dt + \sigma(t, S_t) S_t dW_t, \tag{1}$$

where:

- S_t denotes the stock price at time t ;
- $\mu(t, S_t)$ is the drift function;
- $\sigma(t, S_t)$ is the volatility function;
- $(W_t)_{t \geq 0}$ is a standard Brownian motion.

The initial condition is

$$S_0 = S(0) > 0. \quad (2)$$

To simulate this process numerically, the interval $[0, T]$ is partitioned into N equal time steps:

$$0 = t_0 < t_1 < \dots < t_N = T, \quad \Delta t = \frac{T}{N}. \quad (3)$$

3 Brownian Increments and Normal Random Variables

A key fact used in both implemented methods is that Brownian motion has independent Gaussian increments:

$$W_{t_{j+1}} - W_{t_j} \sim \mathcal{N}(0, \Delta t). \quad (4)$$

Equivalently, if $\xi_j \sim \mathcal{N}(0, 1)$ is a standard normal random variable, then

$$\sqrt{\Delta t} \xi_j \sim \mathcal{N}(0, \Delta t). \quad (5)$$

This is exactly how Brownian increments are generated in the code: I sample standard normal random variables and multiply them by $\sqrt{\Delta t}$.

4 Implemented Drift and Volatility Functions

To satisfy the requirement of using different functions μ and σ , I implemented the following cases.

4.1 Drift Functions

1. Constant drift The first drift function is constant:

$$\mu(t, S) = \mu_0. \quad (6)$$

This is the standard Black–Scholes assumption.

2. Time-varying drift The second drift function varies sinusoidally with time:

$$\mu(t, S) = \mu_0 + A_\mu \sin(2\pi f_\mu t), \quad (7)$$

where:

- μ_0 is the base drift;
- A_μ is the amplitude;
- f_μ is the frequency.

This allows the average trend of the stock price to vary over time.

4.2 Volatility Functions

1. Constant volatility The first volatility function is constant:

$$\sigma(t, S) = \sigma_0. \quad (8)$$

2. Time-varying volatility The second volatility function also varies sinusoidally:

$$\sigma(t, S) = \sigma_0 (1 + A_\sigma \sin(2\pi f_\sigma t)). \quad (9)$$

In the implementation, this expression is bounded from below by a small positive number in order to avoid zero or negative volatility values:

$$\sigma(t, S) = \max\left(\sigma_0 (1 + A_\sigma \sin(2\pi f_\sigma t)), 10^{-8}\right). \quad (10)$$

This is a numerical safeguard and ensures that the simulation remains well-defined.

5 Algorithm 5.1 and Its One-Dimensional Form

The reference algorithm in the book is Algorithm 5.1 for simulating a stock price path in the Black–Scholes model. In the multidimensional notation of the book, the update is written in vector form. Since my implementation simulates a single stock price process, the relevant case is the one-dimensional version.

For one dimension, the algorithm reduces to the update

$$S(t_{j+1}) = S(t_j) \exp\left(\left(\mu_j - \frac{1}{2}\sigma_j^2\right)\Delta t + \sigma_j Z_j\right), \quad (11)$$

where

$$Z_j \sim \mathcal{N}(0, \Delta t). \quad (12)$$

Using the representation

$$Z_j = \sqrt{\Delta t} \xi_j, \quad \xi_j \sim \mathcal{N}(0, 1), \quad (13)$$

the same update can be written as

$$S(t_{j+1}) = S(t_j) \exp\left(\left(\mu_j - \frac{1}{2}\sigma_j^2\right)\Delta t + \sigma_j \sqrt{\Delta t} \xi_j\right). \quad (14)$$

This formula is the basis of the second simulation method implemented in my code.

6 Implemented Simulation Methods

My code includes two simulation methods.

6.1 Method 1: Euler–Maruyama Scheme

The first method is the Euler–Maruyama discretization of the stochastic differential equation

$$dS_t = \mu(t, S_t) S_t dt + \sigma(t, S_t) S_t dW_t. \quad (15)$$

The update formula is

$$S_{j+1} = S_j + \mu(t_j, S_j) S_j \Delta t + \sigma(t_j, S_j) S_j \sqrt{\Delta t} \xi_j, \quad \xi_j \sim \mathcal{N}(0, 1). \quad (16)$$

This method is easy to implement and applies directly to general drift and volatility functions. In the code, the update is written as

```
dS = mu_val * S_i * dt + sigma_val * S_i * np.sqrt(dt) * Z[:, i]
paths[:, i + 1] = np.maximum(S_i + dS, 1e-8)
```

The lower bound 10^{-8} is used to prevent negative or zero prices due to discretization error.

6.2 Method 2: Book Algorithm 5.1 (Exponential Scheme)

The second method is the book-style exponential update inspired by Algorithm 5.1. At each step, the stock price is updated by

$$S_{j+1} = S_j \exp \left(\left(\mu_j - \frac{1}{2} \sigma_j^2 \right) \Delta t + \sigma_j \sqrt{\Delta t} \xi_j \right), \quad \xi_j \sim \mathcal{N}(0, 1), \quad (17)$$

where

$$\mu_j = \mu(t_j, S_j), \quad \sigma_j = \sigma(t_j, S_j). \quad (18)$$

This method is especially important because:

- for constant μ and σ , it coincides with the exact one-step geometric Brownian motion formula;
- it is the method that most closely follows Algorithm 5.1 from the book;
- it preserves positivity automatically, since the exponential is always strictly positive.

In the code, the update is implemented as

```
paths[:, i + 1] = S_i * np.exp(
    (mu_val - 0.5 * sigma_val**2) * dt
    + sigma_val * np.sqrt(dt) * Z[:, i]
)
```

For time-varying $\mu(t, S)$ and $\sigma(t, S)$, the implementation uses left-point evaluation at each step, meaning that the values $\mu(t_j, S_j)$ and $\sigma(t_j, S_j)$ are computed from the current time and current stock price before updating to the next time level.

7 Structure of the Simulation Code

The numerical part of the project is implemented in the file `simulation.py`. The main components are:

7.1 Input Validation

Before starting any simulation, the function

```
_validate_inputs(S0, T, n_steps, n_paths)
```

checks that:

- $S_0 > 0$,
- $T > 0$,
- $N > 0$,
- the number of simulated paths is positive.

7.2 Time Grid

The time grid is created using

```
dt = T / n_steps  
time_grid = np.linspace(0, T, n_steps + 1)
```

which corresponds to the partition

$$t_j = j\Delta t, \quad j = 0, 1, \dots, N. \quad (19)$$

7.3 Storage of Paths

The simulated trajectories are stored in an array of shape

$$(n_paths, n_steps + 1). \quad (20)$$

Thus:

- each row corresponds to one simulated path;
- each column corresponds to one time point.

All paths start from the same initial value:

```
paths[:, 0] = S0
```

7.4 Random Variables

The random variables are generated by

```
Z = rng.standard_normal((n_paths, n_steps))
```

so each entry is an independent standard normal variable.

7.5 Time Iteration

At each time step, the code:

1. reads the current time t_i and the current prices S_i ;
2. evaluates $\mu(t_i, S_i)$ and $\sigma(t_i, S_i)$;
3. updates the stock prices according to the chosen simulation method.

8 Computed Statistics

After simulating the paths, I compute summary statistics using the function

```
compute_path_statistics(paths, time_grid)
```

The following quantities are computed at each time point:

- mean,
- standard deviation,
- median,
- 5th percentile,
- 95th percentile.

More precisely, if $S^{(1)}(t_j), \dots, S^{(M)}(t_j)$ are the simulated values at time t_j , then:

$$\text{mean}(t_j) = \frac{1}{M} \sum_{m=1}^M S^{(m)}(t_j), \quad (21)$$

$$\text{std}(t_j) = \sqrt{\frac{1}{M} \sum_{m=1}^M (S^{(m)}(t_j) - \text{mean}(t_j))^2}, \quad (22)$$

$$\text{median}(t_j) = \text{middle value of the sample at time } t_j. \quad (23)$$

These statistics help describe both the central tendency and the dispersion of the simulated stock prices.

9 Interactive Streamlit Application

The visualization part of the project is implemented in `app.py` using Streamlit and Plotly.

The application allows the user to choose interactively:

- the simulation scheme:
 - Euler–Maruyama,
 - Book Algorithm 5.1 (Exponential);
- the initial price S_0 ;
- the time horizon T ;

- the number of time steps N ;
- the number of simulated paths;
- the random seed;
- the drift function;
- the volatility function.

9.1 Displayed Graphs

The app contains three main tabs.

1. Price Paths This tab displays:

- a subset of simulated trajectories;
- the mean path;
- the median path.

It gives a visual representation of possible future stock-price evolutions under the selected model and simulation scheme.

2. Terminal Distribution This tab shows a histogram of the terminal values

$$S(T). \tag{24}$$

It also displays summary measures such as:

- mean terminal value,
- median terminal value,
- 5th percentile,
- 95th percentile.

3. Statistics This tab shows:

- the evolution over time of the mean and median;
- the evolution over time of the standard deviation.

This makes it possible to study how the average stock price and its dispersion develop through time.

10 Remarks on the Two Methods

The two simulation schemes have different roles.

10.1 Euler–Maruyama

The Euler–Maruyama method is a direct numerical discretization of the stochastic differential equation. It is flexible and conceptually simple, especially when μ and σ are allowed to depend on time and on the current stock price.

10.2 Book Algorithm 5.1

The exponential scheme is the method that best matches the algorithm stated in the assignment. In the classical Black–Scholes model with constant coefficients, it is the exact one-step formula. For this reason, it is the most natural implementation when the goal is to follow Algorithm 5.1 as closely as possible.

10.3 Why both methods were included

I chose to implement both methods because this makes the project more complete:

- the Euler–Maruyama scheme provides a general numerical approximation framework;
- the exponential scheme corresponds more directly to the Black–Scholes formula from the book.

The Streamlit application allows the user to select either method and compare the resulting simulations.

11 Conclusion

In this project I implemented a stock-price simulator in the Black–Scholes framework using two numerical schemes: Euler–Maruyama and a book-style exponential update corresponding to Algorithm 5.1 in one dimension.

The implementation supports both constant and time-varying drift and volatility functions, computes empirical summary statistics, and visualizes the results through an interactive Streamlit interface.

Therefore, the project fulfills the requirement of Exercise 4 by simulating the stock price in the Black–Scholes model for different choices of μ and σ , while also allowing a comparison between a general SDE discretization method and the algorithmic form presented in the reference text.